



भारतीय सूचना प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Information Technology Guwahati

CS236 - ARTIFICIAL INTELLIGENCE LAB
ASSIGNMENT - 10

Problem Statement

Write a Python program to solve the **Tic-Tac-Toe** game using the **minimax algorithm**. The program should allow a human player to play against an AI that uses the minimax algorithm to make optimal moves. The program must:

1. Accept user input to choose whether to play as X or O.
2. Implement the minimax algorithm to determine the AI's moves, ensuring optimal play.
3. Display the game board after each move in a user-friendly format.
4. Handle user input for moves, including validation for invalid or occupied cells.
5. Detect when the game ends and announce the winner or declare a draw.
6. Include a function `get_best_move(board, player)` that returns the best move for a given player on a given board state, which will be tested separately.

Algorithm Specifications

1. Game Representation:

- i. Represent the Tic-Tac-Toe board as a 3×3 grid using a list of lists, with cells containing 'X', 'O', or ' ' (a single space for empty).
- ii. Use 0-based indexing internally, but accept 1-based indexing (rows and columns numbered 1 to 3) for user input.

2. Minimax Algorithm:

- i. Implement the minimax algorithm to evaluate all possible game outcomes recursively.
- ii. Assign scores: +1 for a win by X, -1 for a win by O, and 0 for a draw.
- iii. X is always the **maximizing** player and O is always the **minimizing** player, regardless of whether the human or AI is playing as that symbol. Both players are assumed to play optimally.

3. Game Flow:

- i. Prompt the user to choose their symbol (X or O). If the user chooses X, the user moves first; if the user chooses O, the AI moves first as X.
- ii. Alternate between human and AI moves until the game ends.
- iii. Display the updated board after every move and check for a win or draw after each move.

4. User Input:

- i. Prompt the user to enter their move as "row column" (e.g., "1 2" for row 1, column 2).
- ii. Validate input to ensure:
 - i. Exactly two integers are entered.
 - ii. Both values are within bounds (1 to 3).
 - iii. The chosen cell is empty.
- iii. Re-prompt on invalid input with an appropriate error message.

5. Best Move Function:

- i. Implement `get_best_move(board, player)` that returns the best move as a tuple (row, col) using 0-based indexing, where player is either 'X' or 'O'.
- ii. If multiple moves are equally optimal, return the one with the **smallest row index**, and among those, the **smallest column index** (i.e., scan the board row by row, left to right, and return the first move that achieves the best minimax score).
- iii. You may assume the board is not in a terminal state when this function is called.

Test Cases for `get_best_move`

Ensure that `get_best_move(board, player)` returns the correct move for the following test cases. All coordinates are 0-based. With the tie-breaking rule specified above, each test case has exactly one correct answer.

1. Test Case 1:

```
Board:
X | X |
-----
O | O |
-----
  |  |
Player: X
Expected move: (0, 2)    # Completes X's top row for an immediate win
```

2. Test Case 2:

```
Board:
X | O |
-----
X | O |
```

```

-----
|  | 0
Player: X
Expected move: (2, 0)  # Completes X's left column for an immediate win

```

3. Test Case 3:

```

Board:
X | X |
-----
| 0 |
-----
|  | 0
Player: 0
Expected move: (0, 2)  # Blocks X's immediate win on the top row

```

Sample Interaction

Below is a sample interaction. Note that the AI plays **optimally** at every step — your implementation should produce AI moves consistent with the minimax algorithm, which may differ from this sample depending on the human's choices. Both players playing optimally will always result in a draw.

```

Welcome to Tic-Tac-Toe!
Do you want to play as X or O? X
You are X, I am O.
Current board:
|  |
-----
|  |
-----
|  |
Enter your move (row column): 2 2
You played at (2,2)
Current board:
|  |
-----
| X |
-----
|  |
I play at (1,1)
Current board:
0 |  |
-----
| X |
-----
|  |
Enter your move (row column): 1 3

```

You played at (1,3)

Current board:

```
0 |   | X
```

```
-----
```

```
  | X |
```

```
-----
```

```
  |   |
```

I play at (3,1)

Current board:

```
0 |   | X
```

```
-----
```

```
  | X |
```

```
-----
```

```
0 |   |
```

Enter your move (row column): 2 1

You played at (2,1)

Current board:

```
0 |   | X
```

```
-----
```

```
X | X |
```

```
-----
```

```
0 |   |
```

I play at (2,3)

Current board:

```
0 |   | X
```

```
-----
```

```
X | X | 0
```

```
-----
```

```
0 |   |
```

Enter your move (row column): 3 2

You played at (3,2)

Current board:

```
0 |   | X
```

```
-----
```

```
X | X | 0
```

```
-----
```

```
0 | X |
```

I play at (1,2)

Current board:

```
0 | 0 | X
```

```
-----
```

```
X | X | 0
```

```
-----
```

```
0 | X |
```

Enter your move (row column): 3 3

You played at (3,3)

Current board:

```
0 | 0 | X
```

```
-----  
X | X | O  
-----  
O | X | X  
It's a draw!
```